Discrete Applied Mathematics 379 (2026) 675–684

Contents lists available at ScienceDirect

Discrete Applied Mathematics

journal homepage: www.elsevier.com/locate/dam

Multiple parallel-batch machines scheduling with additive resource assignment and machine available times

Liang Zhang [a], Shisheng Li [b], Zhichao Geng [a,*], Renxia Chen [b], Jiarui Xu [a]

[a] School of Mathematics and Statistics, Zhengzhou University, Zhengzhou, 450001, Henan, China
[b] School of Mathematics and Information Science, Zhongyuan University of Technology, Zhengzhou, 450007, Henan, China

ARTICLE INFO

Article history:
Received 5 April 2025
Received in revised form 10 July 2025
Accepted 28 September 2025

Keywords:
Scheduling
Parallel-batch
Machine available times
Resource assignment
Approximation algorithm

ABSTRACT

In this paper we study a scheduling problem on multiple bounded parallel-batch (or p-batch) machines to minimize makespan. Each machine processes jobs in batches and has a machine-dependent available time, i.e., not all machines are available simultaneously. The processing time of a batch is equal to the largest processing time of the jobs in it. Multiple kinds of additive resources are given and each of them is consumed by at most one job. Each job consumes exactly one kind of resource and its actual processing time depends on the consumed resource. Our main result is presenting an approximation algorithm with the worst-case bound being $2-1/b$ for $m=1$, $2-2/(2b+1)$ for $m=2$ and $2-1/(m-1)b$ for $m\geq 3$, and showing the tightness of this bound, where m is the number of machines and b is the batch capacity.

1. Introduction

Problem description and background We are given s kinds of resources $\mathcal{R}=\{R_1,\ldots,R_s\}$ and n jobs $\mathcal{J}=\{J_1,\ldots,J_n\}$ to be processed on m identical p-batch machines $M_1,\ldots,M_m$, where $s\geq n$ and $m\geq 1$. Each kind of resource is consumed by at most one job, and each job consumes exactly one kind of resource when processed. Denote by $p_{j,k}>0$ the actual processing time of J_j when it consumes resource R_k. Each machine non-preemptively processes jobs in batches, and the processing time of a batch is defined as the largest processing time of the jobs in it. Let b be the batch capacity, which means that a batch contains at most b jobs, and let $b<n$ indicate the bounded case. Assume that machine M_i is available at time $a_i\geq 0$ and the machines are indexed such that $a_1\leq\cdots\leq a_m$. In this paper, by a schedule we mean that each job is assigned a kind of resource, all jobs are partitioned into batches, and the formed batches are scheduled on the machines in some order. Our goal is to find a feasible (satisfying the constraints on the resource assignment, batch capacity and machine available times) schedule to minimize makespan $C_{\max}$, i.e., the maximum completion time of all jobs. Following the three-field notation, this problem can be denoted by $P,a_i|\text{p-batch},b<n,\mathcal{R}|C_{\max}$, where a_i is omitted if all machines are available at time zero.

Our studied problem originates from the following application scenario. In a small-and-medium-sized handicraft enterprise, workers are commonly required to be multi-skilled, that is, they have to be capable of making different kinds of handicrafts. For different workers, the time spent on the same task varies due to the skill difference. For the deciders, to accomplish a set of tasks, they need to assign the workers (as resources) to the tasks sensibly and determine the processing order of the tasks. In this paper we study the problem of integrating scheduling and resource assignment in the setting of multiple p-batch machines.

* Corresponding author.
E-mail address: zcgeng@zzu.edu.cn (Z. Geng).

https://doi.org/10.1016/j.dam.2025.09.035
0166-218X/

Related work and our contributions As a classical (strongly NP-hard) problem, the parallel machines scheduling to minimize makespan has been studied extensively. [9] gave a $(2-1/m)$-approximation algorithm, namely, the well-known List Scheduling (LS) algorithm which assigns the job by an arbitrary job list to the earliest available machine, where m is the number of machines. Later, Graham [10] gave a $(4/3-1/3m)$-approximation algorithm, called Largest Processing Time first (LPT) algorithm, which merely replaces any job list in LS algorithm by the non-increasing list of the processing times. Hochbaum and Shmoys [11] presented a polynomial-time approximation scheme (PTAS), and Alon et al. [1],Jansen [12],Jansen et al. [13] presented improved PTASs, respectively. When m is fixed, Sahni [27] devised a fully polynomial-time approximation scheme (FPTAS). When machines have different available times, Lee [16] showed that the worst-case bound of LPT algorithm is $(3/2-1/2m)$ and gave an improved approximation algorithm with the worst-case bound being $5/4$ for $m=2$ by Lin et al. [21] and $4/3$ for $m\geq 3$ by Lee [16], and Li et al. [18] devised a PTAS and an FPTAS for the cases of fixed m and arbitrary m, respectively.

We proceed to review some work on p-batch scheduling, which have a wide range of applications in semiconductor manufacturing [28], automobile gear manufacturing [8], healthcare [25], etc. For the problem on a single bounded p-batch machine to minimize makespan, Lee [17] gave a polynomial-time algorithm, called Full Batch Longest Processing Time (FBLPT) algorithm, which implies that the corresponding problems on multiple p-batch machines with (or without) different machine available times are equivalent to the counterparts on parallel machines with (or without) different machine available times. When the jobs have release dates, Brucker et al. [5] showed that the problem is strongly NP-hard, and Liu and Yu [23] showed that the problem is NP-hard in the ordinary sense even if there are only two distinct release dates. For the same problem, Deng et al. [6] presented a PTAS, and Poon and Zhang [26], Li et al. [20], Ou et al. [24] proposed improved PTASs. For the problem on multiple p-batch machines to minimize makespan with release dates, Liu et al. [22] presented two approximation algorithms and Li et al. [19] devised a PTAS. For more work on p-batch scheduling, see the survey of Fowler and Mönch [7].

To the best of our knowledge, there is little work on the scheduling with additive resource assignment. For the single-machine problem (with additive resource assignment) to minimize maximum lateness, Barketau et al. [4] proved the strongly NP-hardness. For the problem on two parallel machines to minimize makespan, Bandyapadhyay et al. [2] gave a PTAS and a randomized FPTAS. For the parallel machines problem to minimize makespan with the constraint of the number of jobs on each machine not exceeding a machine-dependent constant, Kress et al. [15] showed the strong NP-hardness and gave two heuristics. For the problem on multiple parallel dedicated machines to minimize makespan, Barketau et al. [3] showed the strong NP-hardness and gave a m-approximation algorithm. They also showed that no $(2-\epsilon)$-approximation algorithm exists when m is arbitrary.

In this paper we address a scheduling problem with additive resource assignment and different machine available times to minimize makespan in the setting of multiple bounded p-batch machines. Our problem is strongly NP-hard when m is arbitrary since it contains problem $P||C_{\max}$ as its special case. Due to the strongly NP-hardness of this problem, we give an approximation algorithm with the worst-case bound being $2-1/b$ for $m=1$, $2-2/(2b+1)$ for $m=2$ and $2-1/(m-1)b$ for $m\geq 3$, and show the tightness of this bound by some instances. Besides, we point out that the worst-case bound can reduce to $2-1/b$ for $m=1$ and $2-2/(mb+1)$ for $m\geq 2$ when our designed algorithm is applied to the case of all available times being zero.

The remainder of this paper is organized as follows. In Section 2 we describe the approximation algorithm and analyze the worst-case bound. In Section 3 we present some numerical results. In the last section, we make some conclusions and give suggestions for future research.

2. Algorithms and analysis

To better depict the resource constraint in the studied problem, we construct a bipartite graph $G=(V,E)$, in which the vertex set is $V=\mathcal{J}\cup\mathcal{R}$ and the edge set is $E=\{e_{j,k}: J_j\in\mathcal{J}, R_k\in\mathcal{R}\}$. From the view of the graph theory, the one-to-one like constraint on the resource assignment is equivalent to find a *n-cardinality matching* (relevant notions can be seen in the book of Korte and Vygen [14]) in G. This implies that the set of the jobs' actual processing times in a feasible schedule exactly corresponds to an n-cardinality matching in G. Define a weight function ω on the edge set E, where the weight $\omega_{j,k}$ of each edge $e_{j,k}$ is set to be $p_{j,k}$.

To proceed, some useful notation are introduced as follows.

- $G_{j,k}$: the graph obtained from G by deleting the vertexes J_j and R_k, their incident edges and all the edges with their weights larger than $\omega_{j,k}$.
- $\mathcal{E}$: the set of the index pair (j,k) satisfying that there is an $(n-1)$-cardinality matching in the graph $G_{j,k}$.
- $M_{j,k}$: the n-cardinality matching in G, which is obtained by adding the edge $e_{j,k}$ to an $(n-1)$-cardinality minimum weight matching in the graph $G_{j,k}$, where $(j,k)\in\mathcal{E}$.
- $\bar{j}(\sigma)$: the index of the resource consumed by job J_j in a schedule σ.
- $p(B)=\max_{J_j\in B} p_{j,\bar{j}(\sigma)}$: the processing time of a batch B in a schedule σ.

An approximation algorithm for problem $P, a_i|$p-batch, $b<n, \mathcal{R}|C_{\max}$ is described as follows.

Before analyzing this algorithm, we first present the following lemma, which reveals an important property of the FBLPT rule and was demonstrated by the Lemma 1 in [24].

Algorithm 1

1. For each $(j,k) \in \mathcal{E}$, construct a schedule $\sigma(j,k)$ in the following way:
   - 1.1 Use Hungarian's algorithm (Kuhn, 1955) to obtain an $(n-1)$-cardinality minimum weight matching in the graph $G_{j,k}$, and then add the edge $e_{j,k}$ to it to generate an n-cardinality matching $M_{j,k}$ in the graph G.
   - 1.2 Partition the jobs into batches by the FBLPT rule: group every b jobs with the smallest indexes in the non-increasing sequence of the jobs' actual processing times (equal to the weights of the corresponding edges in the matching $M_{j,k}$) into batches, then form the last batch with any remaining jobs.
   - 1.3 Regard each batch as a job and run the LPT rule to schedule them on the machines: first sort the batches in the non-increasing order of the batches' processing times, and then schedule each batch on the earliest available machine.
2. Output the schedule with minimum makespan in $\{\sigma(j,k) : (j,k) \in \mathcal{E}\}$.

Lemma 1. *Assume that the jobs are indexed in the non-increasing order of their (fixed) processing times $p_1, \ldots, p_n$. If jobs are partitioned into batches $B_1, \ldots, B_{\lceil \frac{n}{b} \rceil}$ by FBLPT rule with $p(B_1) \geq \cdots \geq p(B_{\lceil \frac{n}{b} \rceil})$, then*

(i) $p(B_l) \leq \frac{1}{b} \sum_{j=(l-2)b+2}^{(l-1)b+1} p_j$ *for* $l = 2, \ldots, \lceil \frac{n}{b} \rceil$;

(ii) $\sum_{l=1}^{\lceil \frac{n}{b} \rceil} p(B_l) \leq (1 - \frac{1}{b}) \max_{1 \leq j \leq n} p_j + \frac{1}{b} \sum_{j=1}^{n} p_j$.

Theorem 1. *Algorithm 1 is an α-approximation algorithm for problem $P, a_i|p\text{-batch}, b < n, \mathcal{R}|C_{\max}$ with a time requirement of $O(sn(s+n)^3)$, where $\alpha = 2 - 1/b$ for $m = 1$, $\alpha = 2 - 2/(2b+1)$ for $m = 2$ and $\alpha = 2 - 1/(m-1)b$ for $m \geq 3$.*

Proof. Let π be an optimal schedule. Without loss of generality, assume that the jobs in π are grouped into $\lceil \frac{n}{b} \rceil$ batches by FBLPT rule by their actual processing times. Let J_u be the job with the largest processing time in π, i.e., $p_{u,\bar{u}(\pi)} = \max_{1 \leq j \leq n} p_{j,\bar{j}(\pi)}$.

Claim 1. $(u, \bar{u}(\pi)) \in \mathcal{E}$.

Proof of Claim 1. Let M be the n-cardinality matching in G that corresponds to the set of the jobs' actual processing times in π. By the suppositions, $e_{u,\bar{u}(\pi)}$ is the edge with the largest weight in M, which implies that any edge $e_{j,k} \in G$ with $w_{j,k} > w_{u,\bar{u}(\pi)}$ is not in M. So, $M \backslash \{e_{u,\bar{u}(\pi)}\}$ is an $(n-1)$-cardinality matching in the graph $G_{u,\bar{u}(\pi)}$ (it is obtained from G by deleting the vertexes J_u and $R_{\bar{u}(\pi)}$, their incident edges and all the edges with the weights larger than $w_{u,\bar{u}(\pi)}$), and naturally there is also an $(n-1)$-cardinality minimum weight matching in $G_{u,\bar{u}(\pi)}$. Further, by the meanings of the set $\mathcal{E}$ and the n-cardinality matching $M_{u,\bar{u}(\pi)}$, we derive $(u, \bar{u}(\pi)) \in \mathcal{E}$. Then Claim 1 follows.

By Claim 1 and Algorithm 1, $\sigma(u, \bar{u}(\pi)) \in \{\sigma(j,k) : (j,k) \in \mathcal{E}\}$ and J_u is also the job with the largest processing time in the schedule $\sigma(u, \bar{u}(\pi))$. For convenience, we simply write $\sigma(u, \bar{u}(\pi))$ as σ^* later, and re-index the jobs such that $p_{1,\bar{1}(\sigma^*)} \geq \cdots \geq p_{n,\bar{n}(\sigma^*)}$ and $u = 1$. Let $B_1, \ldots, B_{\lceil \frac{n}{b} \rceil}$ be the batches in σ^* satisfying $p(B_1) \geq \cdots \geq p(B_{\lceil \frac{n}{b} \rceil})$. This implies that each batch B_l exactly contains the jobs $J_{(l-1)b+1}, \ldots, J_{lb}$ for $l = 1, \ldots, \lceil \frac{n}{b} \rceil - 1$ and the batch $B_{\lceil \frac{n}{b} \rceil}$ contains the jobs $J_{(\lceil \frac{n}{b} \rceil - 1)b+1}, \ldots, J_n$ by the FBLPT rule in Step 1.2 in Algorithm 1. Denote by B_v with $1 \leq v \leq \lceil \frac{n}{b} \rceil$ the batch that completes latest in σ^*, and assume its starting time to be t, which implies $C_{\max}(\sigma^*) = t + p(B_v)$.

If $v = 1$, then we have

$$\min_{(j,k) \in \mathcal{E}} C_{\max}(\sigma(j,k)) \leq C_{\max}(\sigma^*) = a_1 + p(B_1) = a_1 + p_{1,\bar{1}(\pi)} \leq C_{\max}(\pi),$$

which implies that the schedule output by Algorithm 1 is also an optimal schedule. Hereafter, assume $v \geq 2$. Before showing the worst-case bound of Algorithm 1, we first establish a lower bound on the makespan of the optimal schedule π, by using the actual processing times of the jobs in σ^*.

Claim 2. $C_{\max}(\pi) \geq \max\{\frac{1}{m}(\sum_{i=1}^{m} a_i + \frac{1}{b} \sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}),\ a_1 + p_{1,\bar{1}(\sigma^*)}\}$.

Proof of Claim 2. For simplicity, we can assume that there is at least one job on each machine in the optimal schedule π. Otherwise, deleting those machines with no jobs on them in π keeps the makespan of π unchanged and does not decrease the makespan of a schedule output by any algorithm. Let $B'_1, \ldots, B'_{\lceil \frac{n}{b} \rceil}$ be the batches in π. Then, obviously,

$$C_{\max}(\pi) \geq \max\{\frac{1}{m}(\sum_{i=1}^{m} a_i + \sum_{l=1}^{\lceil \frac{n}{b} \rceil} p(B'_l)),\ a_1 + \max_{J_j \in \mathcal{J}} p_{j,\bar{j}(\pi)}\}.$$

Since $p(B'_l) = \max_{J_j \in B'_l}\{p_{j,\bar{j}(\pi)}\} \geq \frac{1}{b}\sum_{J_j \in B'_l}\{p_{j,\bar{j}(\pi)}\}$, we have

$$\sum_{l=1}^{\lceil \frac{n}{b} \rceil} p(B'_l) \geq \frac{1}{b}\sum_{l=1}^{\lceil \frac{n}{b} \rceil}\sum_{J_j \in B'_l} p_{j,\bar{j}(\pi)} = \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\pi)},$$

and so,

$$\begin{aligned} C_{\max}(\pi) &\geq \max\{\frac{1}{m}(\sum_{i=1}^{m} a_i + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\pi)}),\ a_1 + \max_{J_j \in \mathcal{J}} p_{j,\bar{j}(\pi)}\} \\ &= \max\{\frac{1}{m}(\sum_{i=1}^{m} a_i + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\pi)}),\ a_1 + p_{1,\bar{1}(\pi)}\}, \end{aligned} \tag{1}$$

where the last equality holds since J_1 is the job with the largest processing time in π as supposed previously.

By Claim 1, Step 1.1 in Algorithm 1 and the previous suppositions, $M_{1,\bar{1}(\pi)}$ is an n-cardinality matching in the graph G, which corresponds to the set of the jobs' actual processing times in σ^*, and $M_{1,\bar{1}(\pi)}\backslash\{e_{1,\bar{1}(\pi)}\}$ is an $(n-1)$-cardinality minimum weight matching in the graph $G_{1,\bar{1}(\pi)}$. Note that the edge $e_{1,\bar{1}(\pi)}$ is simultaneously in the two n-cardinality matchings which correspond to the sets of the jobs' actual processing times in π and σ^*, respectively. This implies that job J_1 consumes resource $R_{\bar{1}(\pi)}$ and has an identical processing time in π and σ^*, i.e., $p_{1,\bar{1}(\pi)} = p_{1,\bar{1}(\sigma^*)}$. Thus, we have

$$\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\pi)} = \sum_{J_j \in \mathcal{J}\backslash\{J_1\}} p_{j,\bar{j}(\pi)} + p_{1,\bar{1}(\pi)} \geq \sum_{J_j \in \mathcal{J}\backslash\{J_1\}} p_{j,\bar{j}(\sigma^*)} + p_{1,\bar{1}(\sigma^*)} = \sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)},$$

and so,

$$C_{\max}(\pi) \geq \max\{\frac{1}{m}(\sum_{i=1}^{m} a_i + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}),\ a_1 + p_{1,\bar{1}(\sigma^*)}\}$$

holds by (1). Then Claim 2 follows.

To analyze the worst-case bound of Algorithm 1, we begin by considering the case of $m = 1$, i.e., there is only one p-batch machine. Then, we have $C_{\max}(\sigma^*) = a_1 + \sum_{l=1}^{\lceil \frac{n}{b} \rceil} p(B_l)$, and so,

$$\begin{aligned} C_{\max}(\sigma^*) &= a_1 + \sum_{l=1}^{\lceil \frac{n}{b} \rceil} p(B_l) \\ &\leq a_1 + (1-\frac{1}{b})\max_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)} + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)} \ \text{(by Lemma 1(ii))} \\ &\leq (a_1 + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}) + (1-\frac{1}{b})(a_1 + p_{1,\bar{1}(\sigma^*)}) \\ &\leq C_{\max}(\pi) + (1-\frac{1}{b})C_{\max}(\pi) \ \text{(by Claim 2)} \\ &\leq (2-\frac{1}{b})C_{\max}(\pi). \end{aligned}$$

Next, assume $m \geq 2$, and distinguish the following two cases according to whether the start time of batch B_v is earlier than the completion time of batch B_1 in σ^* or not (note that B_1 is on machine M_1 and completes at time $a_1 + p(B_1)$ in σ^* by Algorithm 1).

Cases 1. $t < a_1 + p(B_1)$.

From Step 1.3 in Algorithm 1, the machines $M_2, \ldots, M_m$ are busy before and at time t in σ^* unless they are still unavailable. So we have

$$(m-1)t + p(B_v) \leq \sum_{i=2}^{m} a_i + \sum_{l=2}^{\lceil \frac{n}{b} \rceil} p(B_l). \tag{2}$$

Further, we can deduce

$$\begin{aligned}
C_{\max}(\sigma^*) &= t + p(B_v) \\
&\le \frac{1}{m-1}\Big(\sum_{i=2}^{m} a_i + \sum_{l=2}^{\lceil \frac{n}{b} \rceil} p(B_l) - p(B_v)\Big) + p(B_v) \ \text{(by (2))} \\
&= \frac{1}{m-1}\Big(\sum_{i=1}^{m} a_i + \sum_{l=1}^{\lceil \frac{n}{b} \rceil} p(B_l)\Big) + \frac{m-2}{m-1}p(B_v) - \frac{a_1 + p(B_1)}{m-1} \\
&\le \frac{1}{m-1}\Big(\sum_{i=1}^{m} a_i + (1-\frac{1}{b})\max_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)} + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}\Big) \\
&\qquad + \frac{m-2}{m-1}p(B_v) - \frac{a_1 + p(B_1)}{m-1} \ \text{(by Lemma 1(ii))} \\
&\le \frac{1}{m-1}\Big(\sum_{i=1}^{m} a_i + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}\Big) + \frac{((m-2)b-1)p_{1,\bar{1}(\sigma^*)}}{(m-1)b} - \frac{a_1}{m-1} \\
&\le \frac{1}{m-1}\Big(\sum_{i=1}^{m} a_i + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}\Big) + \frac{((m-2)b-1)p_{1,\bar{1}(\sigma^*)}}{(m-1)b} \\
&\le \frac{m}{m-1}C_{\max}(\pi) + \frac{(m-2)b-1}{(m-1)b}C_{\max}(\pi) \ \text{(by Claim 2)} \\
&= (2 - \frac{1}{(m-1)b})C_{\max}(\pi),
\end{aligned}$$

where the third inequality holds since $\max_{J_j \in \mathcal{J}}\{p_{j,\bar{j}(\sigma^*)}\} = p_{1,\bar{1}(\sigma^*)} = p(B_1) \ge p(B_v)$ by Algorithm 1 and the previous suppositions.

Cases 2. $t \ge a_1 + p(B_1)$.

From Step 1.3 in Algorithm 1, some machines may still be unavailable until immediately before time t in σ^*. Since the available times satisfy $a_1 \le \cdots \le a_m$ as supposed, we can assume that only the machines $M_1, \ldots, M_q$ with $1 \le q \le m$ have been available before time t in σ^*. In fact, each machine M_i of them is busy in the time interval $[a_i, t]$ in σ^*. For $i = 1, \ldots, q$, let $B_1^{(i)}, \ldots, B_{\gamma_i}^{(i)}$ be the batches on M_i among the batches $B_1, \ldots, B_{v-1}$, where γ_i is the number of such batches and $\sum_{i=1}^{q} \gamma_i = v - 1$. Then, we have

$$a_i + \sum_{k=1}^{\gamma_i} p(B_k^{(i)}) \ge t, \quad \text{for } i = 1, \ldots, q. \tag{3}$$

For $i = 1, \ldots, q$ and $k = 1, \ldots, \gamma_i$, let $l(i,k)$ be the index of the batch $B_k^{(i)}$ in the batches $B_1, \ldots, B_{v-1}$, i.e., $B_k^{(i)} = B_{l(i,k)}$ for some $l(i,k)$ with $1 \le l(i,k) \le v-1$. From Lemma 1(i), for each batch $B_k^{(i)}$ with $2 \le l(i,k) \le v-1$, it holds that

$$p(B_k^{(i)}) = p(B_{l(i,k)}) \le \frac{1}{b}\sum_{j=(l(i,k)-2)b+2}^{(l(i,k)-1)b+1} p_{j,\bar{j}(\sigma^*)},$$

where the job $J_{(l(i,k)-1)b+1} \in B_k^{(i)}$ is on machine M_i but the jobs $J_{(l(i,k)-2)b+2}, \ldots, J_{(l(i,k)-1)b} \notin B_k^{(i)}$ may be not on M_i in σ^*. This is followed by

$$\sum_{k=1}^{\gamma_i} p(B_k^{(i)}) \le \frac{1}{b}\sum_{k=1}^{\gamma_i}\sum_{j=(l(i,k)-2)b+2}^{(l(i,k)-1)b+1} p_{j,\bar{j}(\sigma^*)} \ \text{ for } i = 2, \ldots, q$$

and

$$b\sum_{i=2}^{q}\sum_{k=1}^{\gamma_i} p(B_k^{(i)}) \le \sum_{i=2}^{q}\sum_{k=1}^{\gamma_i}\sum_{j=(l(i,k)-2)b+2}^{(l(i,k)-1)b+1} p_{j,\bar{j}(\sigma^*)}. \tag{4}$$

Note that each batch $B_k^{(1)}$ on machine M_1 is exactly composed of the jobs $J_{(l(1,k)-1)b+1}, \ldots, J_{l(1,k)b}$. For each batch $B_k^{(1)}$, write the index $(l(1,k)-1)b+1$ of the job $J_{(l(1,k)-1)b+1}$ as k' simply. Then, the processing time of such a batch $B_k^{(1)}$ is

$p(B_k^{(1)}) = \max_{J_j \in B_k^{(1)}} p_{j,\bar{j}(\sigma^*)} = p_{k',\bar{k}'(\sigma^*)}$ by Algorithm 1 and the previous suppositions, and so,

$$\sum_{k=1}^{\gamma_1} p(B_k^{(1)}) = \sum_{k=1}^{\gamma_1} p_{k',\bar{k}'(\sigma^*)}. \tag{5}$$

Combining (4) and (5) yields

$$\sum_{k=1}^{\gamma_1} p(B_k^{(1)}) + b\sum_{i=2}^{q}\sum_{k=1}^{\gamma_i} p(B_k^{(i)}) \le \sum_{k=1}^{\gamma_1} p_{k',\bar{k}'(\sigma^*)} + \sum_{i=2}^{q}\sum_{k=1}^{\gamma_i}\sum_{j=(l(i,k)-2)b+2}^{(l(i,k)-1)b+1} p_{j,\bar{j}(\sigma^*)}.$$

Note that all the jobs in the batches $B_1, \ldots, B_{v-1}$ of σ^*, with the exception of at least the $b-1$ jobs in $B_{v-1} \setminus \{J_{(v-2)b+1}\}$ (i.e., jobs $J_{(v-2)b+2}, \ldots, J_{(v-1)b}$), appear in the two terms on the right in this inequality, and each job appears exactly once. Thus,

$$\begin{aligned}
&\sum_{k=1}^{\gamma_1} p(B_k^{(1)}) + b\sum_{i=2}^{q}\sum_{k=1}^{\gamma_i} p(B_k^{(i)}) \\
&\le \sum_{l=1}^{v-1}\sum_{J_j\in B_l} p_{j,\bar{j}(\sigma^*)} - \sum_{J_j \in B_{v-1}\setminus\{J_{(v-2)b+1}\}} p_{j,\bar{j}(\sigma^*)} \\
&\le \sum_{l=1}^{v-1}\sum_{J_j\in B_l} p_{j,\bar{j}(\sigma^*)} - (b-1)p(B_v),
\end{aligned} \tag{6}$$

where the last inequality holds since in σ^* the actual processing time of each job $J_j \in B_{v-1} \setminus \{J_{(v-2)b+1}\}$ satisfies $p_{j,\bar{j}(\sigma^*)} \ge p(B_v)$.

Further, we deduce that

$$\begin{aligned}
&b\sum_{i=1}^{m} a_i + \sum_{J_j\in\mathcal{J}} p_{j,\bar{j}(\sigma^*)} \\
&\ge b\sum_{i=1}^{m} a_i + \sum_{l=1}^{v}\sum_{J_j\in B_l} p_{j,\bar{j}(\sigma^*)} \\
&= b\sum_{i=1}^{m} a_i + \sum_{l=1}^{v-1}\sum_{J_j\in B_l} p_{j,\bar{j}(\sigma^*)} + \sum_{J_j\in B_v} p_{j,\bar{j}(\sigma^*)} \\
&\ge b\sum_{i=1}^{m} a_i + \sum_{l=1}^{v-1}\sum_{J_j\in B_l} p_{j,\bar{j}(\sigma^*)} + p(B_v) \\
&\ge b\sum_{i=1}^{m} a_i + \sum_{k=1}^{\gamma_1} p(B_k^{(1)}) + b\sum_{i=2}^{q}\sum_{k=1}^{\gamma_i} p(B_k^{(i)}) + bp(B_v) \quad \text{(by (6))} \\
&= ba_1 + \sum_{k=1}^{\gamma_1} p(B_k^{(1)}) + b\sum_{i=2}^{q}(a_i + \sum_{k=1}^{\gamma_i} p(B_k^{(i)})) + b\sum_{i=q+1}^{m} a_i + bp(B_v) \\
&\ge a_1 + \sum_{k=1}^{\gamma_1} p(B_k^{(1)}) + b\sum_{i=2}^{q}(a_i + \sum_{k=1}^{\gamma_i} p(B_k^{(i)})) + b\sum_{i=q+1}^{m} a_i + bp(B_v) \\
&\ge t + (q-1)bt + (m-q)bt + bp(B_v) \quad \text{(by (3), and } t \le a_{q+1} \le \cdots \le a_m) \\
&= ((m-1)b+1)t + bp(B_v).
\end{aligned}$$

Table 1
The actual processing times of jobs.

	Processing time $p_{j,k}$
$j = 1, \ldots, b$ and $k = j$	$p_{j,k} = 1$
	$p_{b+1,b+1} = \epsilon,\ p_{b+1,2} = \frac{b-1}{b}$
$j = 2, \ldots, b$ and $k = j+1$	$p_{j,k} = \frac{b-1}{b}$

Together with Claim 2, we derive

$$\begin{aligned} C_{\max}(\pi) &\geq \max\{\frac{1}{m}(\sum_{i=1}^{m} a_i + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}),\ a_1 + p_{1,\bar{1}(\sigma^*)}\} \\ &\geq \frac{1}{m}(\sum_{i=1}^{m} a_i + \frac{1}{b}\sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}) \\ &= \frac{1}{mb}(\sum_{i=1}^{m} ba_i + \sum_{J_j \in \mathcal{J}} p_{j,\bar{j}(\sigma^*)}) \\ &\geq \frac{((m-1)b+1)t}{mb} + \frac{p(B_v)}{m} \\ &\geq \frac{mb+1}{mb} p(B_v). \quad (\text{since } t \geq a_1 + p(B_1) \geq p(B_1) \geq p(B_v)) \end{aligned} \tag{7}$$

Consequently, we conclude that

$$\begin{aligned} C_{\max}(\sigma^*) &= t + p(B_v) \\ &\leq \frac{mb}{(m-1)b+1} C_{\max}(\pi) - \frac{b}{(m-1)b+1} p(B_v) + p(B_v) \quad (\text{by (7)}) \\ &= \frac{mb}{(m-1)b+1} C_{\max}(\pi) - \frac{(m-2)b+1}{(m-1)b+1} p(B_v) \\ &\leq \frac{mb}{(m-1)b+1} C_{\max}(\pi) + \frac{(m-2)b+1}{(m-1)b+1} \cdot \frac{mb}{mb+1} C_{\max}(\pi) \quad (\text{by (7)}) \\ &= (2 - \frac{2}{mb+1}) C_{\max}(\pi). \end{aligned}$$

According to the above discussions for Case 1 and Case 2 and the fact of $\min_{(j,k)\in\mathcal{E}}\{C_{\max}(\sigma(j,k))\} \leq C_{\max}(\sigma^*)$, the worst-case bound of Algorithm 1 for $m \geq 2$ is $\max\{2 - 1/(m-1)b, 2 - 2/(mb+1)\}$, which equals to $2 - 2/(2b+1)$ for $m = 2$ and $2 - 1/(m-1)b$ for $m \geq 3$.

It remains to show the time requirement of Algorithm 1. For each $(j,k) \in \mathcal{E}$, to construct a schedule $\sigma(j,k)$, Step 1.1 takes $O((s+n)^3)$ time, Step 1.2 takes $O(n \log n)$ time and Step 1.3 takes $O(n)$ time. Since $|\mathcal{E}| \leq sn$, the overall running time is $O(sn(s+n)^3)$. Then the theorem follows. □

Next, we show that the above worst-case bound is tight by some instances. For $m = 1$, consider the following instance: The available time of M_1 is $a_1 = 0$. The numbers of resources and jobs are both $b+1$. Part of the actual processing times $p_{j,k}, j,k \in \{1, \ldots, b+1\}$ is displayed in Table 1 and the others not in the table are all 2. Then, the n-cardinality matching, which corresponds to the jobs' actual processing times in the schedule σ^* (as defined in the proof of Theorem 1), is $\{e_{1,1}, e_{2,3}, \ldots, e_{b,b+1}, e_{b+1,2}\}$. The n-cardinality matching, which corresponds to the jobs' actual processing times in the optimal schedule π, is $\{e_{1,1}, e_{2,2}, \ldots, e_{b+1,b+1}\}$. Note that $C_{\max}(\sigma^*) = (2b-1)/b$ and $C_{\max}(\pi) = 1+\epsilon$. So, $C_{\max}(\sigma^*)/C_{\max}(\pi)$ tends to $2 - 1/b$ when ϵ is small enough.

For $m = 2$, consider the following instance: The available times of M_1 and M_2 are $a_1 = 0$ and $a_2 = 2b/(2b+1)$. The numbers of resources and jobs are both $3b$. Part of the actual processing times $p_{j,k}, j,k \in \{1, \ldots, 3b\}$ is displayed in Table 2 and the others not in the table are all 2. Then, the n-cardinality matching, which corresponds to the jobs' actual processing times in the schedule σ^*, is $\{e_{1,1}, e_{2,2}, \ldots, e_{b,b}, e_{b+1,b+2}, e_{b+2,b+3}, \ldots, e_{3b-1,3b}, e_{3b,b+1}\}$. The n-cardinality matching, which corresponds to the jobs' actual processing times in the optimal schedule π, is $\{e_{1,1}, e_{2,2}, \ldots, e_{3b,3b}\}$. Note that $C_{\max}(\sigma^*) = 4b/(2b+1)$ and $C_{\max}(\pi) = 1+2b\epsilon$. So, $C_{\max}(\sigma^*)/C_{\max}(\pi)$ tends to $2-2/(2b+1)$ when ϵ is small enough.

For $m \geq 3$, consider another instance: The available times of $M_1, \ldots, M_m$ are $a_1 = 0$ and $a_2 = \cdots = a_m = 1-1/(m-1)b$. The numbers of resources and jobs are both mb. Part of the actual processing times is displayed in Table 3 and the others not in the table are all 2. Then, the n-cardinality matching, which corresponds to the jobs' actual processing times in the schedule σ^*, is $\{e_{1,1}, e_{2,2}, \ldots, e_{b,b}, e_{b+1,b+2}, e_{b+2,b+3}, \ldots, e_{mb-1,mb}, e_{mb,b+1}\}$. The n-cardinality matching, which corresponds to the jobs' actual processing times in the optimal schedule π, is $\{e_{1,1}, e_{2,2}, \ldots, e_{mb,mb}\}$. Note that $C_{\max}(\sigma^*) = 2-1/(m-1)b$ and $C_{\max}(\pi) = 1 + (m-1)b\epsilon$. So, $C_{\max}(\sigma^*)/C_{\max}(\pi)$ tends to $2 - 1/(m-1)b$ when ϵ is small enough.

Table 2
The actual processing times of jobs.

	Processing time $p_{j,k}$
$j = 1, \ldots, b$ and $k = j$	$p_{j,k} = \frac{2b}{2b+1}$
	$p_{b+1,b+1} = \frac{1}{2b+1} + 2b\epsilon$, $p_{b+1,b+2} = \frac{2b}{2b+1}$
$j = b+2, \ldots, 3b-1$ and $k = j$	$p_{j,k} = \frac{1}{2b+1}$
$j = b+2, \ldots, 3b-1$ and $k = j+1$	$p_{j,k} = \epsilon$
	$p_{3b,3b} = \frac{1}{2b+1}$, $p_{3b,b+1} = \epsilon$

Table 3
The actual processing times of jobs.

	Processing time $p_{j,k}$
$j = 1, \ldots, b$ and $k = j$	$p_{j,k} = 1$
	$p_{b+1,b+1} = \frac{1}{(m-1)b} + (m-1)b\epsilon$, $p_{b+1,b+2} = 1$
$j = b+2, \ldots, mb-1$ and $k = j$	$p_{j,k} = \frac{1}{(m-1)b}$
$j = b+2, \ldots, mb-1$ and $k = j+1$	$p_{j,k} = \epsilon$
	$p_{mb,mb} = \frac{1}{(m-1)b}$, $p_{mb,b+1} = \epsilon$

Remark 1. For the case of all machines having an identical available time zero, i.e., problem $P|$p-batch, $b < n, \mathcal{R}|C_{\max}$, by a similar argument we can show that the worst-case bound of Algorithm 1 reduces to β, where $\beta = 2 - 1/b$ for $m = 1$ and $\beta = 2 - 2/(mb+1)$ for $m \geq 2$. Also, this bound can be showed to be tight by some instances.

Remark 2. For the case of the unbounded batch capacity, it is optimal to partition the jobs into only one batch and schedule it on the machine with the earliest available time when the processing times of jobs are fixed. Thus, it suffices to find a n-cardinality matching in which the largest weight is minimized for problem $P, a_i|$p-batch, $b = n, \mathcal{R}|C_{\max}$. This can be realized in $O(sn(s+n)^3)$ time by using a slight adjustment of the Hungarian algorithm to find an edge such that its weight is as small as possible and an $(n-1)$-cardinality matching exists in the graph obtained by deleting all the edges with the weights larger than this edge's.

3. Numerical results

In this section, we present some numerical results to show the effectiveness of Algorithm 1. For simplicity, the available times of all machines are set to zero (note that it is reasonable since the influence is negligible when the number of jobs is large). We compare Algorithm 1 with another two algorithms, called Algorithm 2 and Algorithm 3 respectively, where the former is obtained by merely replacing the LPT rule in Step 1.3 in Algorithm 1 by the LS rule, and the latter first directly finds an n-cardinality minimum weight matching in the original graph G and then groups jobs into batches by the FBLPT rule and schedules them on the machines by the LS rule. Three algorithms are coded in Python 3.9 and run on an Intel(R) Core(TM) i5-13400F CPU @ 4.60 GHz with 32 GB RAM in a Windows 10 environment.

To show the dependence of the algorithms on the number m of machines and the batch capacity b, we set $m = 2, 3, 4, 5$ and $b = 1, 2, 3, 4, 5$, and for each pair of m and b, randomly generate $n = 20, 30, 40, 50$ jobs and n possible processing times with a uniform distribution in $[1, 5]$ for each job. The average makespans of the obtained schedules by the algorithms are displayed in Tables 4 and 5 (in which Algorithms 1, 2 and 3 are simplified as A_1, A_2 and A_3, respectively) and Fig. 1 (only for $n = 20$).

Tables 4 and 5 show that the makespan reduces along with the increase of the batch capacity or (and) the number of machines, and our designed algorithm in this paper surpasses the other two algorithms. Fig. 1 shows that the gaps of the average makespans of the schedules obtained by Algorithm 1 and Algorithm 2 are much smaller than the gaps by Algorithm 1 and Algorithm 3 and the gaps by Algorithm 2 and Algorithm 3. This implies that the way in Step 1.1 in Algorithm 1 works well for finding an n-cardinality matching, and using the LS rule or the LPT-LS rule to schedule the batches obtained by grouping by the FBLPT rule does not matter much. This point is consistent with the previous analysis on the worst-case bound.

4. Conclusions and future research

In this paper we study a specific scheduling problem on multiple p-batch machines with additive resource assignment and machine available times. An approximation algorithm with a complete analysis on the worst-case bound for $m = 1, m = 2$ and $m \geq 3$ are presented, some instances are given to show the tightness of the worst-case bound, and some numerical experiments are done to evaluate the performance of the algorithm. There are still some interesting

Table 4
Average makespans of the obtained schedules when $n = 20$ and $n = 30$.

(n, m, b)	A_1	A_2	A_3	(n, m, b)	A_1	A_2	A_3
(20, 2, 1)	21.8	23.57	53.53	(30, 2, 1)	51	51.5	74.1
(20, 2, 2)	20.07	21.93	48.13	(30, 2, 2)	48.6	49.4	81
(20, 2, 3)	23	23	53	(30, 2, 3)	50.1	54.2	75.3
(20, 2, 4)	22.6	22.93	49.4	(30, 2, 4)	47.1	52.3	83.2
(20, 2, 5)	22.13	22.27	53.87	(30, 2, 5)	48.3	52.9	75.3
(20, 3, 1)	14.27	18.27	32.67	(30, 3, 1)	33.5	35.4	46.9
(20, 3, 2)	14.53	16.13	39.6	(30, 3, 2)	32.1	35	52.3
(20, 3, 3)	14.07	14.87	34	(30, 3, 3)	33.8	36.4	52.8
(20, 3, 4)	15	15.27	32.2	(30, 3, 4)	34.5	35.4	48.2
(20, 3, 5)	16.2	16.2	34.47	(30, 3, 5)	32.8	33.2	50.50
(20, 4, 1)	11.4	13.2	27.7	(30, 4, 1)	24.9	26.2	38.2
(20, 4, 2)	11.73	12.2	26.7	(30, 4, 2)	26.4	29.1	37.8
(20, 4, 3)	11.47	11.8	24	(30, 4, 3)	26.9	27.9	37
(20, 4, 4)	11.73	12.93	25.93	(30, 4, 4)	26.3	28.1	40.1
(20, 4, 5)	12.13	12.33	25.83	(30, 4, 5)	26.5	26.8	34.5
(20, 5, 1)	9.53	10.67	22	(30, 5, 1)	21.6	21.8	30.8
(20, 5, 2)	9	10.33	22.67	(30, 5, 2)	20.6	21.8	33.7
(20, 5, 3)	10.73	11.2	21.6	(30, 5, 3)	20.8	22.8	33.4
(20, 5, 4)	9.6	11	20.47	(30, 5, 4)	20.8	21.8	30.6
(20, 5, 5)	8.6	10.27	20.4	(30, 5, 5)	19.7	23.6	32.9

Table 5
Average makespans of the obtained schedules when $n = 40$ and $n = 50$.

(n, m, b)	A_1	A_2	A_3	(n, m, b)	A_1	A_2	A_3
(40, 2, 1)	86.93	86.93	97.07	(50, 2, 1)	133.5	135.5	137.06
(40, 2, 2)	86.4	96	98	(50, 2, 2)	128.67	141.5	145.33
(40, 2, 3)	89.6	92.13	103.87	(50, 2, 3)	138.67	142.67	147.17
(40, 2, 4)	87.2	93.2	108.4	(50, 2, 4)	134.5	137.17	143
(40, 2, 5)	90.93	92.4	100.53	(50, 2, 5)	138.33	139	143
(40, 3, 1)	55.73	63.2	73.47	(50, 3, 1)	92	94.33	95
(40, 3, 2)	57.07	57.87	64.27	(50, 3, 2)	89.33	91.67	94
(40, 3, 3)	59.33	59.73	63.87	(50, 3, 3)	91.83	93.67	98.83
(40, 3, 4)	56.13	64.93	72.8	(50, 3, 4)	92.5	96.33	98.5
(40, 3, 5)	59.67	60.8	72.4	(50, 3, 5)	90.33	92.83	96.07
(40, 4, 1)	44	48.13	51.6	(50, 4, 1)	69	81.33	89.33
(40, 4, 2)	45.6	47.87	56.4	(50, 4, 2)	67.83	71.83	72.67
(40, 4, 3)	42.67	43.6	55.73	(50, 4, 3)	65	66.17	71.17
(40, 4, 4)	42.93	45.47	53.73	(50, 4, 4)	66.83	73.17	77.07
(40, 4, 5)	44.93	46	44.53	(50, 4, 5)	69	74.33	78.1
(40, 5, 1)	35.2	37.73	41.47	(50, 5, 1)	55.83	58.5	61.83
(40, 5, 2)	37.07	38.27	45.73	(50, 5, 2)	54.67	60.17	68.5
(40, 5, 3)	35.33	38.93	40.67	(50, 5, 3)	54.5	60	62.67
(40, 5, 4)	34.67	38.4	44.27	(50, 5, 4)	55.5	56.5	58
(40, 5, 5)	33.87	37.47	43.6	(50, 5, 5)	54	59.67	61.67

problems that can be considered in the future research. For instance, the exact computational complexity of problem $1|\text{p-batch}, b < n, \mathcal{R}|C_{\max}$ is still unknown, and one also can try to design some approximation algorithms with a better worst-case bound for it. Also, it is of interest to extend the model in this paper to other machine settings, e.g., uniform machines.

Acknowledgments

It is dedicated to Professor Fuji Zhang on the occasion of his 88th birthday. The work was supported by the National Natural Science Foundation of China (Grant Nos. 12471305, 12401427 and 12271491) and Key Research Projects of Henan Higher Education Institutions [24A110014].

Data availability

No data was used for the research described in the article.

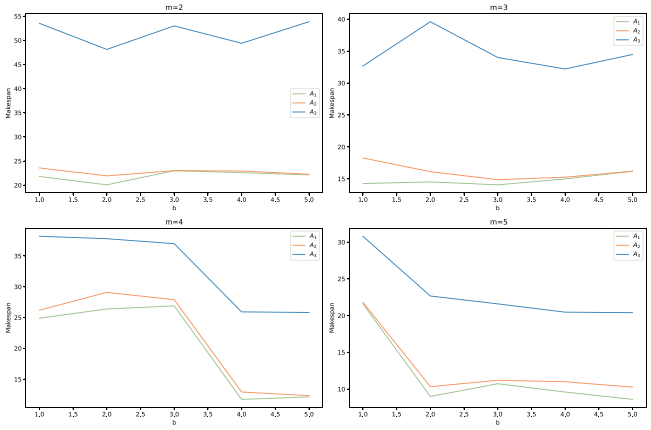

Fig. 1. Average makespans when $n = 20$.

References

[1] N. Alon, Y. Azar, G.J. Woeginger, T. Yadid, Approximation schemes for scheduling on parallel machines, J. Sched. 1 (1998) 55–66.
[2] S. Bandyapadhyay, F. Fomin, T. Inamdar, F. Panolan, K. Simonov, Socially fair matching: Exact and approximation algorithms, in: Algorithms and Data Structures Symposium, Springer, 2023, pp. 79–92.
[3] M. Barketau, E. Pesch, Y. Shafransky, Minimizing maximum weight of subsets of a maximum matching in a bipartite graph, Discrete Appl. Math. 196 (2015) 4–19.
[4] M. Barketau, E. Pesch, Y. Shafransky, Scheduling dedicated jobs with variative processing times, J. Comb. Optim. 31 (2016) 774–785.
[5] P. Brucker, A. Gladky, H. Hoogeveen, M.Y. Kovalyov, C.N. Potts, T. Tautenhahn, S.L. Van De Velde, Scheduling a batching machine, J. Sched. 1 (1998) 31–54.
[6] X.T. Deng, C.K. Poon, Y.Z. Zhang, Approximation algorithms in batch processing, J. Comb. Optim. 7 (2003) 247–257.
[7] J.W. Fowler, L. Mönch, A survey of scheduling with parallel batch (p-batch) processing, European J. Oper. Res. 298 (2022) 1–24.
[8] R. Gokhale, M. Mathirajan, Heuristic algorithms for scheduling of a batch processor in automobile gear manufacturing, Int. J. Prod. Res. 49 (2011) 2705–2728.
[9] R.L. Graham, Bounds for certain multiprocessing anomalies, Bell Syst. Tech. J. 45 (1966) 1563–1581.
[10] R.L. Graham, Bounds on multiprocessing timing anomalies, SIAM J. Appl. Math. 17 (1969) 416–429.
[11] D.S. Hochbaum, D.B. Shmoys, Using dual approximation algorithms for scheduling problems theoretical and practical results, J. ACM 34 (1987) 144–162.
[12] K. Jansen, An EPTAS for scheduling jobs on uniform processors: Using an MILP relaxation with a constant number of integral variables, SIAM J. Discrete Math. 24 (2010) 457–485.
[13] K. Jansen, K.M. Klein, J. Verschae, Improved efficient approximation schemes for scheduling jobs on identical and uniform machines, in: Proceedings of the 13th Workshop on Models and Algorithms for Planning and Scheduling Problems, MAPSP 2017, 2017, pp. 77–79.
[14] B. Korte, J. Vygen, Combinatorial Optimization, Springer Berlin, Heidelberg, 2018.
[15] D. Kress, S. Meiswinkel, E. Pesch, The partitioning min–max weighted matching problem, European J. Oper. Res. 247 (2015) 745–754.
[16] C.Y. Lee, Parallel machines scheduling with nonsimultaneous machine available time, Discrete Appl. Math. 30 (1991) 53–61.
[17] C.Y. Lee, Minimizing makespan on a single batch processing machine with dynamic job arrivals, Int. J. Prod. Res. 37 (1999) 219–236.
[18] W.D. Li, J.B. Li, J.P. Li, T.Q. Zhang, Parallel scheudling problem with non-simultaneous machine available times, J. Systems Sci. Math. Sci. 30 (2010) 433–440.
[19] S.G. Li, G.J. Li, S.Q. Zhang, Minimizing makespan with release times on identical parallel batching machines, Discrete Appl. Math. 148 (2005) 127–134.
[20] S.G. Li, Z.G. Yang, X.Q. Qi, Minimizing makespan in single-machine batch processing, Oper. Res. Trans. 3 (2006) 1–37.
[21] G.H. Lin, Y. He, Y.J. Yao, H.Y. Lu, Exact bounds of the modified LPT algorithms applying to parallel machines scheduling with nonsimultaneous machine available times, Appl. Math.-A J. Chin. Univ. 12 (1997) 109–116.
[22] L.L. Liu, C.T. Ng, T.C.E. Cheng, Scheduling jobs with release dates on parallel batch processing machines to minimize the makespan, Optim. Lett. 8 (2014) 307–318.
[23] Z. Liu, W. Yu, Scheduling one batch processor subject to job release dates, Discrete Appl. Math. 105 (2000) 129–136.
[24] J.W. Ou, L.F. Lu, X.L. Zhong, Parallel-batch scheduling with rejection: Structural properties and approximation algorithms, European J. Oper. Res. 310 (2023) 1017–1032.
[25] O. Ozturk, M.L. Espinouse, M.D. Mascolo, A. Gouin, Makespan minimisation on parallel batch processing machines with non-identical job sizes and release dates, Int. J. Prod. Res. 50 (2012) 6022–6035.
[26] C.K. Poon, P.X. Zhang, Minimizing makespan in batch machine scheduling, Algorithmica 39 (2004) 155–174.
[27] S.K. Sahni, Algorithms for scheduling independent tasks, J. ACM 23 (1976) 116–127.
[28] R. Uzsoy, C.Y. Lee, L.A. Martin-Vega, A review of production planning and scheduling models in the semiconductor industry part i: system characteristics, performance evaluation and production planning, IIE Trans. 24 (1992) 47–60.